9. 트랜잭션 풀 분석

개요

Reth의 트랜잭션 풀은 이더리움 네트워크에서 트랜잭션을 효율적으로 관리하고 블록 생성을 위한 최적의 트랜잭션을 제공하는 핵심 컴포넌트입니다. `EthTransactionPool` 은 제네릭 트랜잭션 풀 구현체로, 메모리 관리, 우선순위 처리, 검증 등의 기능을 제공합니다.

주요 책임

- 들어오는 트랜잭션 기록 및 저장
- 기존 트랜잭션 조회 및 제공
- 블록 생성을 위한 최적 트랜잭션 정렬 및 제공
- 메모리 사용량 모니터링 및 풀 크기 제한 적용
- EIP-4844 blob 트랜잭션의 별도 blob 저장소 관리

핵심 구조체 및 트레이트

EthTransactionPool

```
/// 기본 이더리움 트랜잭션 풀 타입 별칭
pub  type EthTransactionPool<Client, S, T = EthPooledTransaction> = Pool<
TransactionValidationTaskExecutor<EthTransactionValidator<Client, T>>,
CoinbaseTipOrdering<T>,
S,
```

```
>;
```

`EthTransactionPool` 은 다음 컴포넌트들로 구성됩니다:

- **Validator**: `TransactionValidationTaskExecutor` - 비동기 트랜잭션 검증
- **Ordering**: `CoinbaseTipOrdering` - 코인베이스 팁 기반 우선순위 정렬
- **BlobStore**: blob 트랜잭션 데이터 저장소

Pool 구조체

```rust
#[derive(Debug)]
pub struct Pool<V, T: TransactionOrdering, S> {
/// Arc로 래핑된 풀 내부 구조
pool: Arc<PoolInner<V, T, S>>,
}
```

PoolInner 구조체

```rust
pub struct PoolInner<V, T, S>
where
T: TransactionOrdering,
{
/// 주소를 정수로 매핑하는 내부 식별자
identifiers: RwLock<IdentifierMap>,
/// 실제 트랜잭션 풀
```

```
pool: RwLock<TxPool<T>>,

/// 풀 설정

config: PoolConfig,

/// 트랜잭션 상태 변경 이벤트 리스너 관리

event_listener: RwLock<PoolEventBroadcast<T::Transaction>>,

/// 트랜잭션 검증기

validator: V,

/// Blob 저장소

blob_store: S,

}
```

서브풀 시스템

Reth의 트랜잭션 풀은 4개의 서브풀로 구성되어 트랜잭션을 효율적으로 관리합니다.

서브풀 종류

1. Pending Pool (대기 풀)

- **목적**: 다음 블록에 포함될 준비가 된 트랜잭션들
- **조건**:
- nonce 갭이 없음
- 충분한 잔액 보유
- 적절한 가스 수수료
- blob 수수료 충족 (EIP-4844의 경우)

2. Queued Pool (대기열 풀)

- **목적**: 미래 트랜잭션들 (nonce 갭 또는 잔액 부족)
- **조건**:
- nonce 갭 존재
- 잔액 부족
- 선행 트랜잭션 대기 중

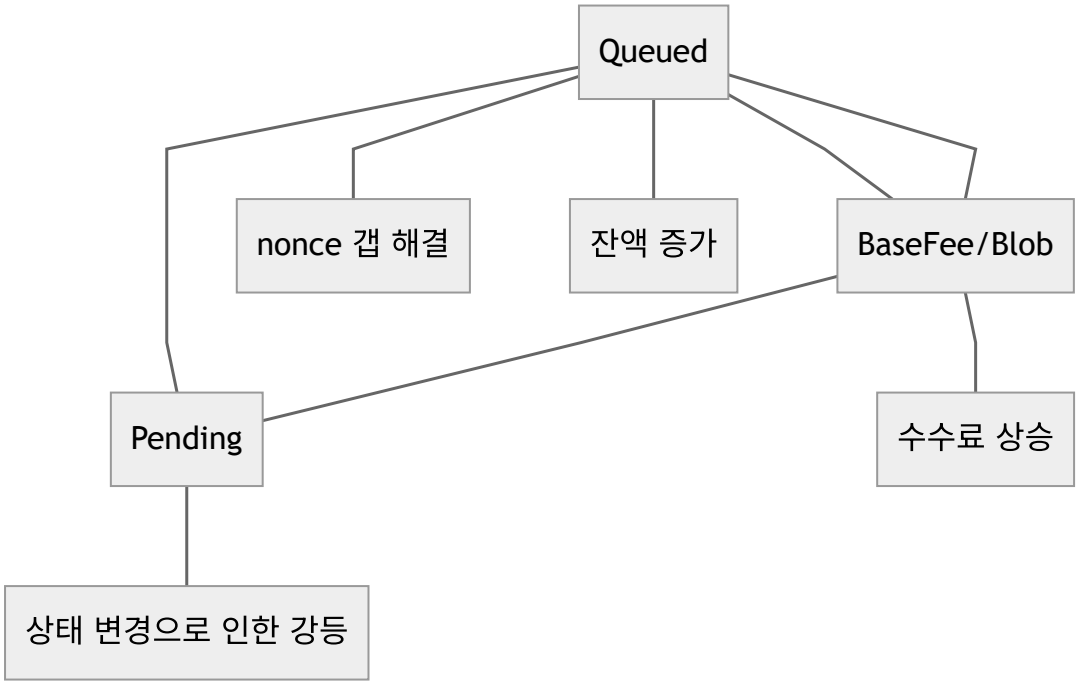
3. BaseFee Pool (기본 수수료 풀)

- **목적**: 현재 기본 수수료보다 낮은 트랜잭션들
- **조건**: 유효하지만 현재 기본 수수료 미달

4. Blob Pool (Blob 풀)

- **목적**: EIP-4844 blob 트랜잭션 중 대기 상태가 아닌 것들
- **조건**: 기본 수수료 또는 blob 수수료 부족

서브풀 상태 전환



트랜잭션 검증 시스템

TransactionValidationTaskExecutor

```
pub struct TransactionValidationTaskExecutor<V> {
/// 별도 태스크에서 트랜잭션을 검증하는 검증기
pub validator: Arc<V>,
/// 검증 태스크로의 채널
to_validation_task: Arc<Mutex<UnboundedSender<ValidationTask<V::Transaction>>>>,
}
```

검증 과정

1. 무상태 검사 (Stateless Checks)

- **트랜잭션 타입**: 포크별 지원 여부 확인
- **크기**: 입력 데이터 ≤ 128KB (기본값)
- **가스**: 한도 ≤ 블록 가스 한도
- **수수료**: 우선순위 수수료 ≤ 최대 수수료
- **체인 ID**: 현재 체인과 일치
- **내재 가스**: 데이터 및 액세스 리스트에 충분한 가스
- **Blob** (EIP-4844): 유효한 개수, KZG 증명

2. 상태 기반 검사 (Stateful Checks)

- **발신자**: 바이트코드 없음 (EIP-7702 위임 제외)

- **Nonce**: ≥ 계정 nonce
- **잔액**: 값 + (가스_한도 × 최대_가스당_수수료) 커버

일반적인 검증 오류

```
// Nonce 관련 오류
NonceNotConsistent // Nonce가 너무 낮음

// 잔액 관련 오류
InsufficientFunds // 잔액 부족

// 가스 관련 오류
ExceedsGasLimit // 가스 한도 초과
Underpriced // 수수료 너무 낮음

// Blob 관련 오류
MissingEip4844BlobSidecar // Blob 사이드카 누락
InvalidEip4844Blob // 잘못된 blob 증명
```

우선순위 처리 및 정렬

CoinbaseTipOrdering

```
pub struct CoinbaseTipOrdering<T>(PhantomData<T>);
```

```rust
impl<T> TransactionOrdering for CoinbaseTipOrdering<T>

where

T: PoolTransaction + 'static,

{

type PriorityValue = u128;

type Transaction = T;



fn  priority(

&self,

transaction: &Self::Transaction,

base_fee: u64,

) -> Priority<Self::PriorityValue> {

transaction.effective_tip_per_gas(base_fee).into()

}

}
```

우선순위 계산

트랜잭션의 우선순위는 **유효 팁 (Effective Tip)**을 기반으로 계산됩니다:

```
effective_tip = min(max_priority_fee_per_gas, max_fee_per_gas - base_fee)
```

- 높은 팁일수록 높은 우선순위
- 기본 수수료를 고려한 실제 마이너 보상 계산

- EIP-1559 동적 수수료 모델 지원

메모리 풀 최적화

PoolConfig 설정

```
pub struct PoolConfig {
/// 대기 서브풀의 최대 트랜잭션 수
pub pending_limit: SubPoolLimit,
/// 기본수수료 서브풀의 최대 트랜잭션 수
pub basefee_limit: SubPoolLimit,
/// 대기열 서브풀의 최대 트랜잭션 수
pub queued_limit: SubPoolLimit,
/// Blob 서브풀의 최대 트랜잭션 수
pub blob_limit: SubPoolLimit,
/// 계정당 보장되는 최대 실행 가능한 트랜잭션 슬롯
pub max_account_slots: usize,
/// 가스 한도
pub gas_limit: u64,
/// 최대 대기열 생존 시간
pub max_queued_lifetime: Duration,
}
```

SubPoolLimit

```rust
pub struct SubPoolLimit {
/// 풀의 최대 트랜잭션 수
pub max_txs: usize,
/// 풀의 최대 결합 크기 (바이트)
pub max_size: usize,
}

impl Default for SubPoolLimit {
fn default() -> Self {
Self {
max_txs: 10_000, // 10,000개 트랜잭션
max_size: 20 * 1024 * 1024, // 20MB
}
}
}
```

메모리 관리 전략

1. **크기 기반 제한**: 각 서브풀별 메모리 사용량 제한
2. **개수 기반 제한**: 트랜잭션 개수 제한
3. **계정별 슬롯**: 계정당 최대 트랜잭션 수 제한 (기본 16개)
4. **생존 시간**: 오래된 트랜잭션 자동 제거
5. **우선순위 기반 축출**: 한도 초과 시 낮은 우선순위 트랜잭션 제거

가스 가격 정책

가격 범프 (Price Bump) 정책

```
pub struct PriceBumpConfig {
/// 기본 가격 범프 (%)
pub default_price_bump: u128, // 10%
/// Blob 트랜잭션 교체 가격 범프 (%)
pub replace_blob_tx_price_bump: u128, // 100%
}
```

교체 정책

- **일반 트랜잭션**: 10% 가격 인상으로 교체 가능
- **Blob 트랜잭션**: 100% 가격 인상 필요 (높은 비용으로 인한 보호)
- **로컬 트랜잭션**: 특별 면제 정책 적용 가능

최소 수수료 정책

```
impl PoolConfig {
/// 최소 프로토콜 기본 수수료 (MIN_PROTOCOL_BASE_FEE)
pub minimal_protocol_basefee: u64,
/// 풀 허용을 위한 최소 우선순위 수수료
pub minimum_priority_fee: Option<u128>,
```

```
}
```

트랜잭션 플로우

1. 네트워크 경로 (P2P)

```
네트워크 피어

↓

Transactions 또는 NewPooledTransactionHashes 메시지

↓

TransactionsManager

↓

pool.add_external_transactions() [Origin: External]

↓

트랜잭션 검증 및 풀 추가
```

2. RPC 경로 (로컬 제출)

```
eth_sendRawTransaction RPC 호출

↓

원시 바이트 디코딩 및 발신자 복구

↓

pool.add_transaction() [Origin: Local]

↓
```

트랜잭션 검증 및 풀 추가

3. 트랜잭션 원본 (Origin)

- **Local**: RPC를 통해 제출된 트랜잭션 (신뢰됨)
- **External**: 네트워크 피어로부터의 트랜잭션 (엄격한 검증)
- **Private**: 네트워크에 전파되지 않는 로컬 트랜잭션

풀 유지보수

블록 업데이트 처리

```
impl<V, T, S> TransactionPoolExt for Pool<V, T, S> {
fn  on_canonical_state_change<B>(&self, update: CanonicalStateUpdate<'_, B>)
where
B: Block,
{
self.pool.on_canonical_state_change(update);
}

fn  update_accounts(&self, accounts: Vec<ChangedAccount>) {
self.pool.update_accounts(accounts);
}
}
```

유지보수 작업

1. **채굴된 트랜잭션 제거**: 새 블록에 포함된 트랜잭션 제거
2. **계정 상태 업데이트**: nonce, 잔액 변경 반영
3. **수수료 업데이트**: 새로운 기본 수수료 및 blob 수수료 적용
4. **서브풀 간 이동**: 상태 변경에 따른 트랜잭션 재배치
5. **크기 제한 적용**: 한도 초과 시 최악의 트랜잭션 제거

Blob 트랜잭션 처리

BlobStore 인터페이스

```
pub trait BlobStore: fmt::Debug + Send + Sync + 'static {
fn insert(&self, tx: TxHash, data: Arc<BlobTransactionSidecarVariant>) -> Resul
fn get(&self, tx: TxHash) -> Result<Option<Arc<BlobTransactionSidecarVariant>>,
fn delete(&self, tx: TxHash) -> Result<(), BlobStoreError>;
}
```

Blob 관리

- **별도 저장**: Blob 데이터는 메인 풀과 분리된 저장소에 보관
- **크기 최적화**: 큰 blob 데이터로 인한 메모리 압박 방지
- **생명주기 관리**: 트랜잭션 제거 시 관련 blob 데이터도 정리

성능 최적화

비동기 검증

```
// 검증 작업을 별도 태스크에서 수행
async fn validate_all(
&self,
origin: TransactionOrigin,
transactions: impl IntoIterator<Item = V::Transaction> + Send,
) -> Vec<TransactionValidationOutcome<V::Transaction>>
```

배치 처리

- **일괄 검증**: 여러 트랜잭션을 한 번에 검증
- **병렬 처리**: 독립적인 검증 작업 병렬 실행
- **캐싱**: 검증 결과 캐싱으로 중복 작업 방지

메트릭 및 모니터링

```
// 풀 크기 모니터링
pub fn pool_size(&self) -> PoolSize {
self.pool.size()
}

// 초과 상태 확인
pub fn is_exceeded(&self) -> bool {
self.pool.is_exceeded()
```

```
}
```

이벤트 시스템

트랜잭션 이벤트

```
pub enum TransactionEvent {
/// 트랜잭션이 풀에 추가됨
Added,
/// 트랜잭션이 풀에서 제거됨
Removed,
/// 트랜잭션이 다른 서브풀로 이동됨
Moved { from: SubPool, to: SubPool },
}
```

리스너 등록

```
// 특정 트랜잭션 이벤트 리스닝
fn transaction_event_listener(&self, tx_hash: TxHash) -> Option<TransactionEven


// 모든 트랜잭션 이벤트 리스닝
fn all_transactions_event_listener(&self) -> AllTransactionsEvents<Self::Transa
```

```
// 새로운 대기 트랜잭션 리스닝

fn pending_transactions_listener_for(&self, kind: TransactionListenerKind) -> R
```

설정 및 커스터마이징

로컬 트랜잭션 설정

```
pub struct LocalTransactionConfig {

/// 로컬 트랜잭션에 면제 적용 안 함

pub no_exemptions: bool,

/// 로컬로 간주될 주소들

pub local_addresses: HashSet<Address>,

/// 로컬 트랜잭션 전파 여부

pub propagate_local_transactions: bool,

}
```

커스터마이징 포인트

1. **검증기 교체**: 사용자 정의 검증 로직
2. **정렬 알고리즘**: 사용자 정의 우선순위 계산
3. **Blob 저장소**: 다양한 저장소 백엔드 지원
4. **이벤트 처리**: 사용자 정의 이벤트 리스너

관련 컴포넌트

의존성 관계

- **ChainSpecProvider**: 체인 사양 정보 제공
- **StateProviderFactory**: 상태 정보 접근
- **TaskSpawner**: 비동기 작업 실행
- **BlobStore**: Blob 데이터 저장

상호작용 방식

- **RPC 서비스**: 트랜잭션 제출 및 조회
- **네트워크 관리자**: P2P 트랜잭션 전파
- **블록 빌더**: 최적 트랜잭션 선택
- **동기화 시스템**: 상태 변경 알림

Reth의 트랜잭션 풀은 높은 처리량과 효율적인 메모리 관리를 통해 이더리움 네트워크의 트랜잭션을 안정적으로 처리하는 핵심 컴포넌트입니다.